Collections in Java

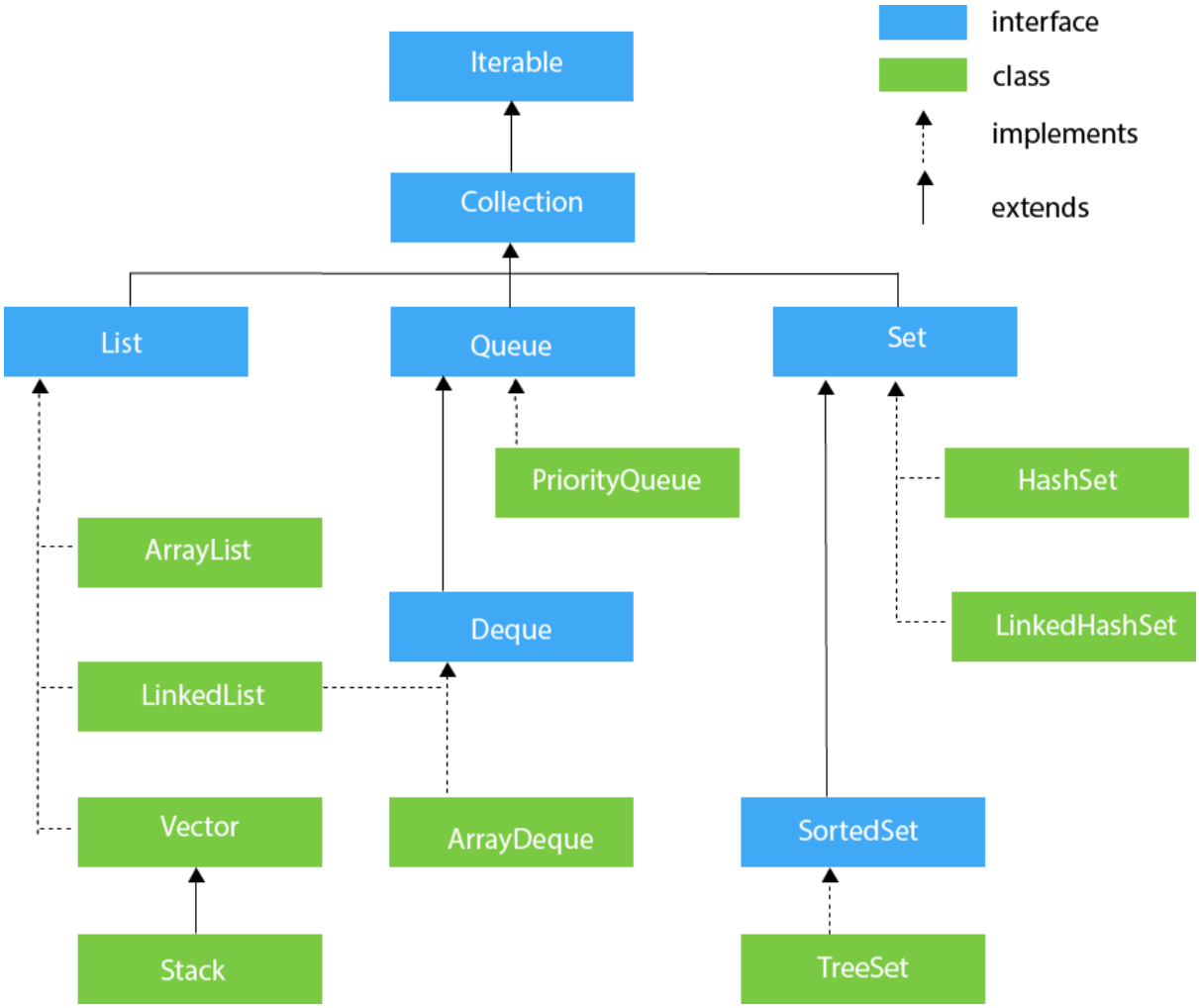
- ✓ The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.
- ✓ Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- ✓ Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).

What is Collection in Java

A Collection represents a single unit of objects, i.e., a group.

What is Collection framework

The Collection framework represents a unified architecture for storing and manipulating a group of objects. It has: Interfaces and its implementations, i.e., classes



Methods of Collection interface

There are many methods declared in the Collection interface. They are as follows:

No.	Method	Description
1	public boolean add(E e)	It is used to insert an element in this collection.
2	public boolean addAll(Collection collection)	It is used to insert the specified collection elements in the invoking collection.
3	public boolean remove(Object obj)	It is used to delete an element from the collection.
4	public boolean removeAll (Collection collection)	It is used to delete all the elements of the specified collection from the invoking collection.
5	public int size()	It returns the total number of elements in the collection.
6	public void clear()	It removes the total number of elements from the collection.
7	public boolean contains(Object element)	It is used to search an element.
8	public boolean containsAll(Collection collection)	It is used to search the specified collection in the collection.
9	public Iterator iterator()	It returns an iterator.
10	public boolean isEmpty()	It checks if collection is empty.
11	public boolean equals(Object element)	It matches two collections.

Iterator interface

Iterator interface provides the facility of iterating the elements in a forward direction only.

Methods of Iterator interface

There are only three methods in the Iterator interface. They are:

No.	Method	Description
1	public boolean hasNext()	It returns true if the iterator has more elements otherwise it returns false.
2	public Object next()	It returns the element and moves the cursor pointer to the next element.
3	public void remove()	It removes the last elements returned by the iterator. It is less used.

what method do you use to obtain an element in the collection from an iterator? explain with example

java.util package has public interface Iterator and contains three methods:

1. boolean hasNext(): It returns true if Iterator has more element to iterate.
2. Object next(): It returns the next element in the collection until the hasNext()method return true. This method throws 'NoSuchElementException' if there is no next element.
3. void remove(): It removes the current element in the collection. This method throws 'IllegalStateException' if this function is called before next( ) is invoked.

In general, to use an iterator to cycle through the contents of a collection, follow these steps –

- ✓ Obtain an iterator to the start of the collection by calling the collection's iterator( ) method.
- ✓ Set up a loop that makes a call to hasNext( ). Have the loop iterate as long as hasNext( ) returns true.
- ✓ Within the loop, obtain each element by calling next( ).

Example:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorDemo {

   public static void main(String args[]) {
      // Create an array list
      List<String> al = new ArrayList<>();
```

```
        // add elements to the array list
        al.add("C");
        al.add("A");
        al.add("E");
        al.add("B");
        al.add("D");
        al.add("F");

        // Use iterator to display contents of al
        System.out.print("Original contents of al: ");
        Iterator<String> itr = al.iterator();

        while(itr.hasNext()) {
            Object element = itr.next();
            System.out.print(element + " ");
        }
        System.out.println();
    }
}
```

List Interface

- ✓ List interface is the child interface of Collection interface.
- ✓ It inhibits a list type data structure in which we can store the ordered collection of objects.
- ✓ It can have duplicate values.
- ✓ List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.

To instantiate the List interface, we must use :

```
List <data-type> list1= new ArrayList();
List <data-type> list2 = new LinkedList();
List <data-type> list3 = new Vector();
List <data-type> list4 = new Stack();
```

ArrayList

- ✓ The ArrayList class implements the List interface.
- ✓ It uses a dynamic array to store the duplicate element of different data types.
- ✓ The ArrayList class maintains the insertion order and is non-synchronized.
- ✓ The elements stored in the ArrayList class can be randomly accessed.

Consider the following example.

```
import java.util.*;
```

```
class TestJavaCollection1{
public static void main(String args[]){
ArrayList<String> list=new ArrayList<String>();
list.add("Ravi");   //Adding object in arraylist
list.add("Vijay");
list.add("Ravi");
list.add("Ajay");
//Traversing list through Iterator
Iterator itr=list.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Output:

```
Ravi
Vijay
Ravi
Ajay
```

LinkedList

- ✓ LinkedList implements the Collection interface.
- ✓ It uses a doubly linked list internally to store the elements. It can store the duplicate elements.
- ✓ It maintains the insertion order and is not synchronized.
- ✓ In LinkedList, the manipulation is fast because no shifting is required.

Consider the following example.

```
import java.util.*;
public class TestJavaCollection2{
public static void main(String args[]){
LinkedList<String> al=new LinkedList<>();
al.add("Ravi");
al.add("Vijay");
al.add("Ravi");
al.add("Ajay");
```

```
Iterator<String> itr=al.iterator();
while(itr.hasNext()){
System.out.println(itr.next());

}
}
}
```

Output:

```
Ravi
Vijay
Ravi
Ajay
```

Vector

- ✓ Vector uses a dynamic array to store the data elements. It is similar to ArrayList.

Following is the list of constructors provided by the vector class.

Sr.No.	Constructor & Description
1	**Vector( )** This constructor creates a default vector, which has an initial size of 10.
2	**Vector(int size)** This constructor accepts an argument that equals to the required size, and creates a vector whose initial capacity is specified by size.
3	**Vector(int size, int incr)** This constructor creates a vector whose initial capacity is specified by size and whose increment is specified by incr. The increment specifies the number of elements to allocate each time that a vector is resized upward.
4	**Vector(Collection c)** This constructor creates a vector that contains the elements of collection c.

Vector defines the following methods –

Sr.No.	Method & Description
1	**boolean add(E e)** This method appends the specified element to the end of this Vector.
2	**int size()** This method returns the number of components in this vector.
3	**void addElement(E obj)** This method adds the specified component to the end of this vector, increasing its size by one.
4	**int capacity()** This method returns the current capacity of this vector.
5	**void clear()** This method removes all of the elements from this vector.
6	**object firstElement()** This method returns the first component (the item at index 0) of this vector.
7	**boolean contains(Object o)** This method returns true if this vector contains the specified element.
8	**boolean containsAll(Collection<?> c)** This method returns true if this Vector contains all of the elements in the specified Collection.
9	**E get(int index)** This method returns the element at the specified position in this Vector.
10	**int hashCode()** This method returns the hash code value for this Vector.
11	**int indexOf(Object o)** This method returns the index of the first occurrence of the specified element in this vector, or -1 if this vector does not contain the element.

12	**void insertElementAt(E obj, int index)** This method inserts the specified object as a component in this vector at the specified index.
13	**boolean isEmpty()** This method tests if this vector has no components.
14	**E lastElement()** This method returns the last component of the vector.
15	**E remove(int index)** This method removes the element at the specified position in this Vector.
16	**boolean removeAll(Collection<?> c)** This method removes from this Vector all of its elements that are contained in the specified Collection.
17	**void removeAllElements()** This method removes all components from this vector and sets its size to zero.
18	**boolean removeElement(Object obj)** This method removes the first occurrence of the argument from this vector.
19	**void removeElementAt(int index)** This method deletes the component at the specified index.
20	**void setSize(int newSize)** This method sets the size of this vector.

Consider the following **example**.

```
import java.util.*;
public class TestJavaCollection3{
public static void main(String args[]){
Vector<String> v=new Vector<String>();
v.add("Ayush");
```

```
v.add("Amit");
v.add("Ashish");
v.add("Garima");
Iterator<String> itr=v.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Output:

```
Ayush
Amit
Ashish
Garima
```

Stack

- ✓ The stack is the subclass of Vector.
- ✓ It implements the last-in-first-out data structure, i.e., Stack.
- ✓ The stack contains all of the methods of Vector class and also provides its methods like boolean push(), boolean peek(), boolean push(object o), which defines its properties.

Consider the following example.

```
import java.util.*;
public class TestJavaCollection4{
public static void main(String args[]){
Stack<String> stack = new Stack<String>();
stack.push("Ayush");
stack.push("Garvit");
stack.push("Amit");
stack.push("Ashish");
stack.push("Garima");
stack.pop();
Iterator<String> itr=stack.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
```

```
}
}
}
```

Output:

```
Ayush
Garvit
Amit
Ashish
```

PriorityQueue

- ✓ The PriorityQueue class implements the Queue interface.
- ✓ It holds the elements or objects which are to be processed by their priorities.
- ✓ PriorityQueue doesn't allow null values to be stored in the queue.

Consider the following example.

```
import java.util.*;
public class TestJavaCollection5{
public static void main(String args[]){
PriorityQueue<String> queue=new PriorityQueue<String>();
queue.add("Amit Sharma");
queue.add("Vijay Raj");
queue.add("JaiShankar");
queue.add("Raj");
System.out.println("head:"+queue.element());
System.out.println("head:"+queue.peek());
System.out.println("iterating the queue elements:");
Iterator itr=queue.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
queue.remove();
queue.poll();
System.out.println("after removing two elements:");
Iterator<String> itr2=queue.iterator();
while(itr2.hasNext()){
System.out.println(itr2.next());
```

```
}
}
}
```

Output:

```
head:Amit Sharma
head:Amit Sharma
iterating the queue elements:
Amit Sharma
Raj
JaiShankar
Vijay Raj
after removing two elements:
Raj
Vijay Raj
```

Set

- ✓ Set Interface in Java is present in java.util package.
- ✓ It extends the Collection interface.
- ✓ It represents the unordered set of elements which doesn't allow us to store the duplicate items.
- ✓ We can store at most one null value in Set. Set is implemented by HashSet, LinkedHashSet, and TreeSet.

Set can be instantiated as:

```
Set<data-type> s1 = new HashSet<data-type>();
Set<data-type> s2 = new LinkedHashSet<data-type>();
Set<data-type> s3 = new TreeSet<data-type>();
```

HashSet

- ✓ HashSet class implements Set Interface.
- ✓ It represents the collection that uses a hash table for storage.
- ✓ Hashing is used to store the elements in the HashSet.
- ✓ It contains unique items.

Consider the following example.

```
import java.util.*;
public class TestJavaCollection7{
```

```
public static void main(String args[]){
//Creating HashSet and adding elements
HashSet<String> set=new HashSet<String>();
set.add("Ravi");
set.add("Vijay");
set.add("Ravi");
set.add("Ajay");
//Traversing elements
Iterator<String> itr=set.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Output:

```
Vijay
Ravi
Ajay
```

LinkedHashSet

- ✓ LinkedHashSet class represents the LinkedList implementation of Set Interface.
- ✓ It extends the HashSet class and implements Set interface.
- ✓ Like HashSet, It also contains unique elements. It maintains the insertion order and permits null elements.

Consider the following example.

```
import java.util.*;
public class TestJavaCollection8{
public static void main(String args[]){
LinkedHashSet<String> set=new LinkedHashSet<String>();
set.add("Ravi");
set.add("Vijay");
set.add("Ravi");
set.add("Ajay");
Iterator<String> itr=set.iterator();
while(itr.hasNext()){
```

```
System.out.println(itr.next());
}
}
}
```

Output:

```
Ravi
Vijay
Ajay
```

SortedSet

- ✓ SortedSet is the alternate of Set interface that provides a total ordering on its elements.
- ✓ The elements of the SortedSet are arranged in the increasing (ascending) order.
- ✓ The SortedSet provides the additional methods that inhibit the natural ordering of the elements.

The SortedSet can be instantiated as:

```
SortedSet<data-type> set = new TreeSet();
```

TreeSet

- ✓ Java TreeSet class implements the Set interface that uses a tree for storage.
- ✓ Like HashSet, TreeSet also contains unique elements.
- ✓ However, the access and retrieval time of TreeSet is quite fast.
- ✓ The elements in TreeSet stored in ascending order.

Consider the following example:

```
import java.util.*;
public class TestJavaCollection9{
public static void main(String args[]){
//Creating and adding elements
TreeSet<String> set=new TreeSet<String>();
set.add("Ravi");
set.add("Vijay");
set.add("Ravi");
set.add("Ajay");
//traversing elements
```

```
Iterator<String> itr=set.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
}
}
```

Output:

```
Ajay
Ravi
Vijay
```

Difference Between List and Set in Java

List Interface	Set Interface
A List interface is an ordered collection of values in which values may or may not be duplicated	A set interface is a collection that does not allow duplicate elements and has no guaranteed order for its elements.
Lists allow duplicate values	Sets can't have duplicate values
Lists can be indexed and positionally accessed	Sets cannot be indexed and cannot be accessed based on their positions
Multiple null elements can be stored	Only one null element can be stored at a time
Examples: ArrayList, LinkedList, Vector, Stack.	**Examples:** HashSet, LinkedHashSet.
The insertion order is maintained by the List.	It doesn’t maintain the insertion order of elements.
List is often used when we need to frequently access the elements using their indices.	It is used when we need to store distinct elements.
The listiterator() is used to iterate the List elements.	The iterator() method is used to iterate Set elements.
Lists are best used when a small dataset is at hand	Sets are best used when a larger dataset needs to be processed.
We can get the element of a specified index from the list using the get() method.	We cannot find the element from the Set based on the index because it doesn't provide any get method().

List Interface

A List interface is an ordered collection of values in which values may or may not be duplicated

Example of List Interface

```
{
        // List declaration
   List<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(5);
        list.add(4);

     System.out.println("List = " + list);
}
Output
List = [1, 2, 3, 5, 4]
```

Set Interface

A set interface is a collection that does not allow duplicate elements and has no guaranteed order for its elements.

Example of Set Interface

```
 // Set declaration
        Set<Integer> s = new HashSet<>();
        s.add(1);
        s.add(2);
        s.add(3);
        s.add(5);
        s.add(4);
```

The **output** for the following code is:

```
Set = [1, 2, 3, 5, 4]
```

Explain List, Map and Set with example.

What is List interface?

- ✓ The Java ***collections interface's sub-interface*** is called the ***List interface***.
- ✓ It offers index-based ways to ***add, update, remove, and search*** for elements.
- ✓ Duplicate elements are also possible, and the null entries can also be kept in the list.
- ✓ Allows duplicate elements.
- ✓ Maintains the order of elements.
- ✓ Supports access by index, e.g., get(index).

Examples:

- List: ArrayList, LinkedList.

Example:

```
import java.util.*;
public class Javalist {
   public static void main(String args[])
   {

      // here create a List
      List<String> al = new ArrayList<>();
```

```
        // Now add the elements in the List
        al.add("BMW");
        al.add("Hundai");
        al.add("Toyota");
        al.add("Swift");

        // Iterating the List
        // element using for-each loop
        for (String cars : al)
            System.out.println(cars);
    }
}
```

Output:

```
BMW
Hundai
Toyota
Swift
```

What is Set?

- ✓ The Set is present in Java and uses an unordered approach.
- ✓ Does not allow duplicate elements.
- ✓ No guaranteed order (except ordered variants like LinkedHashSet).
- ✓ Doesn't support index-based access.

Examples:

- Set: HashSet, TreeSet.

Example:

```
// A Java program to illustrate a Set.
// add Elements using Set.

import java.util.*;
public class JavaSet {

    public static void main(String[] args)
    {
        // Set demonstration via using HashSet
```

```
        Set<String> Set = new HashSet<String>();

        // Adding some Elements
        Set.add("Java");
        Set.add("Python");
        Set.add("DBMS");
        Set.add("Machine Learning");
        Set.add("Operating System");

        // Here Set follows unordered way.
        System.out.println(Set);
    }
}
```

Output:

```
[Java, Operating System, DBMS, Machine Learning, Python]
```

What is Map?

- ✓ Java Map, or java.util.Map, is a java interface.
- ✓ A key and a value are mapped by the term "map".
- ✓ A Java Map is able to hold pairs of keys and values in more detail.
- ✓ An individual value is connected to each key, and the value can later be retrieved using only the key after being saved in a Map.

Example:

```
// A Java program to illustrate a Map.
 // add elements using Map
import java.util.*;
class JavaMap {
   public static void main(String args[])
   {

      // Creating object for a Map.
      Map<String, Integer> map  = new HashMap< String, Integer>();

      // Adding Elements using Map.
      map.put( "Rajat", 101);
      map.put("Shyam", 102);
```

```
        map.put("Rahul", 103);
        map.put("Krishna", 104);
        // here, elements may traverse in any order
        for (Map.Entry m : map.entrySet()) {
          System.out.println(m.getKey() + " "
                    + m.getValue());
        }
    }
}
```

Output:

```
Rahul 103
Shyam 102
Krishna 104
Rajat 101
```

The key differences between List, Set, and Map in Salesforce are:

1. List is an ordered collection of elements that can contain duplicates, Set is an unordered collection of unique elements, and Map is a collection of key-value pairs.
2. List is accessed by index position, Set does not support index position and is accessed by the value itself, and Map is accessed by the key.
3. List maintains order, Set does not maintain order, and Map does not guarantee order.
4. List is useful when you need to maintain order and access elements by their index position, Set is useful when you need to perform operations on unique elements, and Map is useful when you need to access elements by a specific key.

Difference between List, Set, and Map in Java

List	Set	Map
The list interface allows duplicate elements	Set does not allow duplicate elements.	The map does not allow duplicate elements
The list maintains insertion order.	Set do not maintain any insertion order.	The map also does not maintain any insertion order.

List	Set	Map
We can add any number of null values.	But in set almost only one null value.	The map allows a single null key at most and any number of null values.
List implementation classes are Array List, LinkedList.	Set implementation classes are HashSet, LinkedHashSet, and TreeSet.	Map implementation classes are HashMap, HashTable, Tree Map, ConcurrentHashMap, and LinkedHashMap.
The list provides get() method to get the element at a specified index.	Set does not provide get method to get the elements at a specified index	The map does not provide get method to get the elements at a specified index
If you need to access the elements frequently by using the index then we can use the list	If you want to create a collection of unique elements then we can use set	If you want to store the data in the form of key/value pair then we can use the map.
To traverse the list elements by using Listlterator.	Iterator can be used traverse the set elements	Through keyset, value, and entry set.